Phase 3 – Project Report

CMPT 276 – Intro to Software Engineering

Escape from the Burnaby Mountain Prison

Group Members:
Jagdeep Sidhu
Arsh Behniwal
Taranjot Aujla
Dominic Janus
Nick Zhou

March 24, 2026
Simon Fraser University

Contents

1 Introduction **3**

2 Test Classes **3**

3 Guard Testing **3**

4 Reward Testing **5**

5 Sound Testing **5**

6 Hazard and Powerups Testing **6**

7 Game and Player Testing **6**

8 PrisonMap and Panel Testing **7**

1 Introduction

In Phase 3, we focused on systematically testing the core logic and gameplay of our project, using JUnit and Maven. Our objective was to increase the code coverage, but also improve the overall robustness of the game under normal, edge, and failure-prone situations. By implementing 77 automated tests across 8 test classes, we aimed to cover player behaviours, guard logic, hazards, powerups, rewards, map functionality, game transitions, sound, and UI experiences.

2 Test Classes

The first step of our testing process was developing unit tests for components that could be tested independently.

- **TestRewards:** tests reward initialization, updating scores, collecting rewards, and additional tests to ensure the Rewards are functioning as intended.
- **TestHazard:** tests the effect of hazards such as handcuffs, parking tickets, spoiled milk, and a bear to ensure the applicable damaging effects are applied.
- **TestGuard:** tests guard logic including patrol behaviour, chase behaviour, reset logic, spawns, and movement constraints.
- **TestGameAndPlayer:** tests the core game and player logic, including difficulty selection, menu transitions, timing, reset behaviour, duplicate reward handling, and win/loss processing.
- **TestPowerups:** tests that each powerup initializes, correctly handles score management, and applies necessary effects correctly, including edge cases.
- **TestPrisonMapAndPanel:** tests the map's construction, collision handling, coin spawning, reset behavior, and rendering of the panels in the live state.
- **TestSound:** tests that the audio system manages valid and invalid clip loading and playback calls without crashing or interfering with the game.

3 Guard Testing

The unit tests for Guard.java are situated in TestGuard.java and it covers twenty-nine test cases across the main features of the guard AI. The key features tested include, but aren't limited to, the guard construction and initial state (testGuardConstructorSetsPatrolStateForPatrolType, testGuardConstructorSetsChaseStateForChaseType), which verifies the PATROL and CHASE types initialize to the correct state, alert state detection via isAlertState() (testIsAlertStateReturnsTrueWhenChasing, testIsAlertStateReturnsFalseWhenPatrolling), the reset() method (testResetRestoresPositionAndState, testResetRestoresPrecisePosition), which restores the position and state of the guards. Additional tests verify patrol movement which

ensures that the guards stay within walkable tiles (testGuardStaysOnWalkableTilesWhilePatrolling), state transitions (testGuardTransitionsToReturningWhenTooFarFromHome, testGuardReturnsToPatrolStartAfterChasing) between PATROLLING, CHASING, and RETURNING, player damage on contact (testPlayerLosesLifeWhenGuardCatchesThem), an invulnerability window which prevents immediate damage after touching the guard an initial time (testPlayerDoesNotLoseLifeWhileInvulnerable), and zone-based spawning (testSpawnRandomGuardProducesWalkablePosition, testSpawnRandomGuardDoesNotSpawnOnPlayer, testSpawnRandomGuardDoesNotOverlapExistingGuards), which ensures that the guards don't spawn on top of the player or other guards.

There are three integration tests which help us deduce if the cross-component behaviour is working correctly, testPlayerDoesNotLoseLifeWhileInvulnerable, as well as testGuardChasesAndDamagesPlayerMultipleTimesAfterInvulnerabilityExpires. These two tests help us figure out the interactions between Guard and Player, enabling us to confirm that the damage system and invulnerability window are working as expected between both classes. Another test that was conducted was testGameResetSpawnsCorrectNumberOfGuardsForEachDifficulty, which tests the interactions between Guard, Game, and the difficulty system, this test eased the worries we had regarding if startMatch() was correctly spawning guards and transitioning to the PLAYING state for all three difficulty levels.

We were able to automatically run all these tests with Maven by using mvn test from the project root, and in order to only run the guard test, we used mvn test -Dtest=TestGuard. This command allowed us to specifically isolate my changes, as it would grant me the ability to receive direct and focused feedback on exactly what I was working on. JaCoCo was also configured in pom.xml, which generated a coverage report at target/site/jacoco/index.html after each run.

Each test had a single job and responsibility to complete, a descriptive name, as well as an assertion message which explained what failed. A @BeforeEach method was also included, as it enabled me to create a fresh map before each test to prevent shared state issues, and after running all twenty-nine tests for Guard.java, we received a line coverage result of 89% and a branch coverage result of 79%. The remaining uncovered branches were in spawnRandomGuard() and returnToPatrol(). However, the fallback path for spawnRandomGuard() only appeared after two hundred failed spawn attempts, which requires a nearly fully blocked map to trigger, and the issue with returnToPatrol only occurred when the guard was already at the patrol start before returning, as these were defensive fallback paths that were already expected, they weren't particularly prioritized.

The most significant finding that I had discovered was that returnToPatrol() previously had 0% coverage before this phase, meaning the guard's ability to navigate back to its patrol start had never been verified. However, by adding multiple tests, we were able to bring it to full coverage and confirmed that it worked the way we had envisioned. We were also able to clarify that the invulnerability cooldown was moved from Guard into Player.loseLife() thanks to the damage tests. All in all, there weren't any game changing bugs that were found in Guard.java that required us to change any of the initial code as all twenty-nine tests passed.

4 Reward Testing

The unit tests for the reward collection system are primarily implemented in TestRewards.java. The tests cover six cases across the core features of the collection mechanics. The features tested include reward construction and initialization. These tests verify that the objects store their map coordinates and begin in an active state. The applyTo() method was thoroughly tested to ensure each specific reward type correctly applied its unique score value and the temporary speed boost. Moreover, failure scenarios and edge cases were tested to improve robustness. Tests were written to verify that an already inactive reward cannot be collected again for duplicated points, and a player suffering the hands-tied effect is blocked from collecting a reward.

To evaluate whether cross-component behaviour was working correctly, we implemented several integration tests within TestGameAndPlayer.java. Tests such as testDuplicateRewardTypeOnlyCountsTowardsCompletionOnce and testGameTransitionsToLoseAndWinStatesAndPanelRenderOverlays helped test the interactions between Rewards, Player, and the main Game controller. These tests confirmed that rewards only counted towards completion once, even if duplicated, and the game tracks three unique rewards to unlock the exit. These tests also verified that collecting all three rewards and navigating to the exit tile will transition the game into the LEVEL_COMPLETE state and player.reset() successfully resets the reward inventory.

During testing, we found some flags with our instruction coverage. Initially, JaCoCo flagged the switch(RewardType) statement with partial coverage despite testing every enum value. We discovered this was due to the missing default statement. By adding the default statements, we achieved 100% instruction coverage and 83% branch coverage for the Rewards.java class. The JaCoCo report partially flagged the switch statements for not covering the added default case where an unknown reward type was passed into the setter method and applyTo method. In practice, this branch is unreachable because RewardTypes is strictly defined as an enum, preventing invalid values from being passed.

5 Sound Testing

The unit tests for the audio system are implemented in TestSound.java, covering three test cases that focus on stability, initialization, and exception handling. The core features tested include bypassing a null audio clip to avoid system crashes/ glitches, and boundary and error handling with testValidIndex(0), confirming that requesting an unmaped audio clip safely aborts the operation and leaves the clip as null. Lastly, testPlayback() verifies the core functionality of the sound class by loading a valid audio file, asserting the clip object initializes successfully, and all playback methods execute without throwing exceptions.

Adding an integration, testAudio() in TestGameAndPlayer.java, the test helped evaluate cross-component behaviour and confirm the Game class can safely interact with the Sound objects. By designing our test cases to target cases of failure, such as null audio clips, we achieved 97% line and 100% branch coverage. The testing phase ultimately proved that our defensive programming prevents missing files or overlapping audio requests from blocking state changes or crashing the main game loop thread.

6 Hazard and Powerups Testing

For the game's item economy, we divided our testing strategy into two distinct layers to ensure absolute reliability. Firstly, we developed foundational Unit Tests (TestHazard.java, TestPowerups.java) to test all items in isolation. These tests verify that all the items initialize at the correct, valid map coordinates, apply the proper score increase/decrease based on their Enum types and successfully toggle their active states when their applyTo() methods are called.

Once the isolated logic was verified, we tested how these items interacted with the core game engine. We utilized Integration Tests (TestHazardsAndPowerups.java) that simulated the 30 FPS game loop. These tests actively validated and checked the complex cross-component interactions between the Player and items. For instance, one test checked if the Handcuffs debuff physically blocked the Player from picking up a Powerup, which it did, functioning perfectly within a live game environment.

By combining isolated unit tests with integration tests, we achieved exceptional coverage for the item economy. According to our JaCoCo metrics, the Hazard and Powerups classes reached 99-100% Line Coverage with 0 missed lines. During testing, we successfully verified that our most complex edge cases, such as preventing a player from using a deactivated item, worked exactly as intended without requiring any structural changes to our Phase 2 production code.

7 Game and Player Testing

The unit tests for the core game loop and player systems are primarily situated in TestGameAndPlayer.java, and it covers eleven test cases across the main gameplay systems that were added and refined during Phase 2 and Phase 3. The key features tested include difficulty selection and clamping, ensuring that the player begins with the correct number of lives on Easy, Medium, and Hard, pause and resume behaviour through the Escape key, menu, story, and pause screen mouse interactions, player hit invulnerability and the red flash timing, reset behaviour, duplicate reward handling, win and lose state transitions, as well as several stability tests for game startup, restart behaviour, audio calls, and thread interruption.

Several of these were integration tests rather than pure unit tests. For example, testMenuAndPauseMouseClicksRouteToExpectedStates and testGameTransitionsToLoseAndWinStatesAndPanelRendersOverlays helped us verify the interactions between Game, Player, MapPanel, and the reward progression system. Another important test was testPlayerResetClearsCollectedRewardsAndTemporaryHudState, which allowed us to confirm that resetting the player correctly cleared temporary effects, collected reward state, and other HUD-facing status information. During this process, we were able to identify that reset logic needed to fully clear some temporary player state, which was then corrected in the production code.

We were able to run these tests with Maven by using mvn test from the project root, and to isolate only the game and player test cases, we used mvn test -Dtest=TestGameAndPlayer. JaCoCo was also configured in pom.xml, which generated a coverage report at target/site/jacoco/index.html after each run. According to the JaCoCo report, Game.java achieved 78.0% line coverage and 74.0% branch coverage, while Player.java achieved 96.9% line cov-

erage and 76.5% branch coverage. The lower branch coverage in Game.java was expected because the class contains several state transitions, audio branches, and loop-related paths that are more difficult to fully exhaust in unit tests. Even so, the testing phase gave us strong confidence that the most important gameplay and player-facing systems were functioning as intended

8 PrisonMap and Panel Testing

The unit tests for the map and rendering systems are implemented in TestPrisonMapAndPanel.java, and it covers ten test cases across the core features of the map layout, collision logic, reset behaviour, and rendering pipeline. The key features tested include map initialization, scaled row and column counts, correct placement of the start and end tiles, the start cell layout with walls, doorway, and furniture decorations, safe out of bounds queries, player collision handling against walls, coin spawning behaviour, coin reset behaviour, and live rendering checks for the panel. These tests helped ensure that both the logical map state and the visual panel state remained consistent with one another.

To evaluate whether the cross-component behaviour was working correctly, we also included integration style tests that connected PrisonMap, MapPanel, Player, and Game together. For instance, testMapPanelInitializationTracksMapStateAndRendersBaseScene verified that the panel could initialize correctly, track guard spawns, and render the base scene without throwing exceptions. Another important integration test was testGameStartMatchRespawnsCoinsAndMapPanelRendersLiveGameState, which confirmed that starting a match through the Game class correctly respawned the expected number of coins and that the panel could still render the updated game state safely. These tests were especially useful because a large portion of our map and panel work involved the camera, terrain drawing, decorations, and HUD-related rendering, all of which depend on several classes working together rather than in isolation.

We were able to automatically run these tests with Maven by using mvn test, and to isolate the map and panel tests we used mvn test -Dtest=TestPrisonMapAndPanel. According to the JaCoCo report, PrisonMap.java achieved 97.6% line coverage and 95.5% branch coverage, while MapPanel.java achieved 73.6% line coverage and 61.5% branch coverage. The lower coverage for MapPanel.java was expected, since much of that class is made up of rendering branches, overlay states, and UI drawing paths that are more difficult to fully automate. However, the tests still covered the most important gameplay-facing rendering paths, and no game-breaking bugs were discovered in the map or panel systems once the tests were completed.